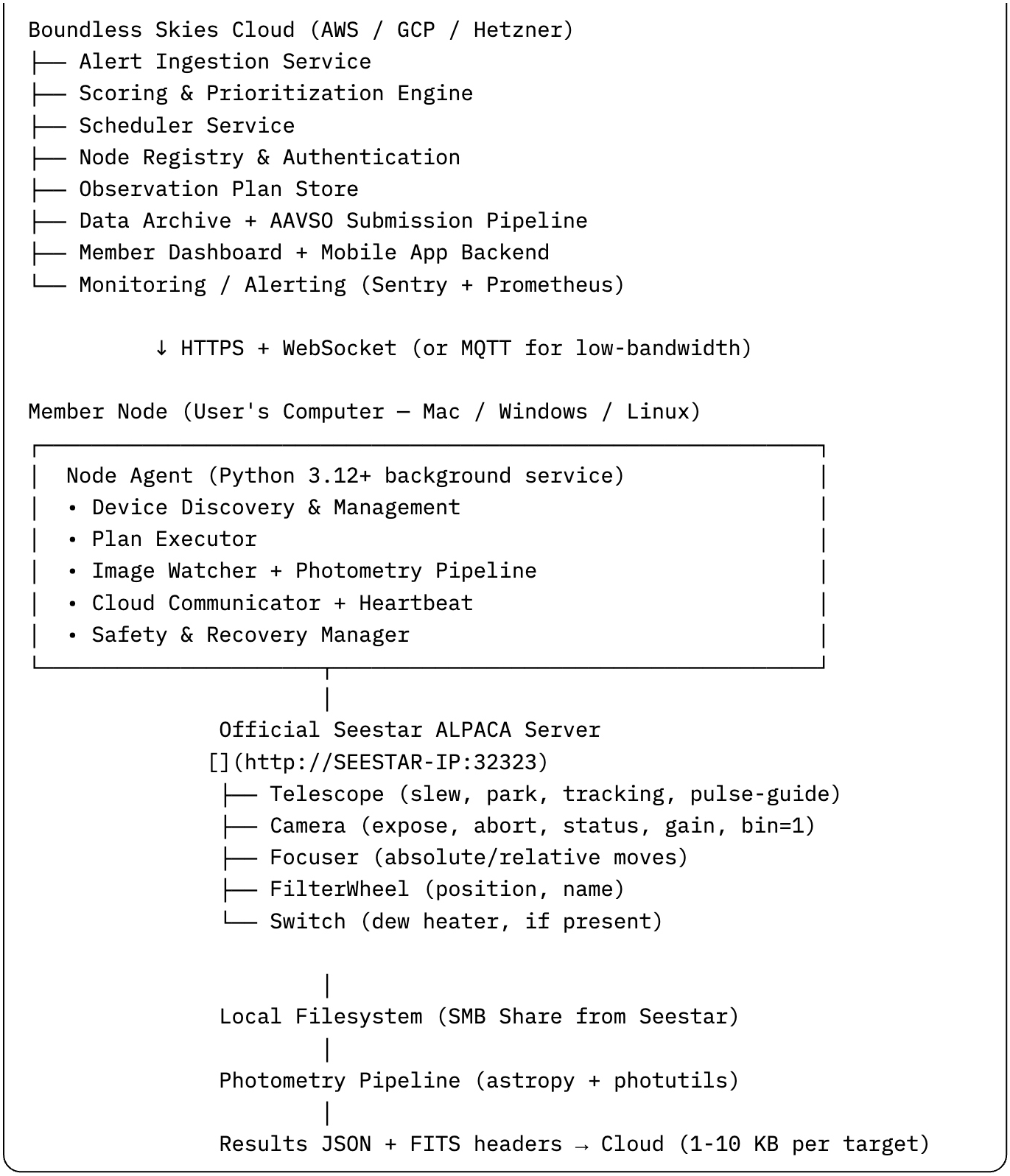
Boundless Skies — Full Technical Stack Architecture (Detailed Design)

This is a production-grade, expert-level design for the **entire technical stack**, built around **custom Python software** + **official ZWO Seestar ALPACA** (100% safe, no `seestar_alp`, no third-party wrappers that could brick devices).

It is **fully cross-platform** from day one (macOS native for you, Windows for most members, Linux/RPi later). It follows the original Boundless Skies PDF as closely as possible while being realistic for 2026 hardware/firmware.

1. High-Level System Overview

```text
```



2. Core Components — Deep Dive

A. Cloud Backend (FastAPI + PostgreSQL + Redis)

Tech Stack:

- **Framework**: FastAPI (async, excellent for WebSocket/MQTT)
- **Database**: PostgreSQL (with PostGIS for location-based queries)
- **Queue**: Redis + Celery (or RQ) for alert processing and scheduling
- **Storage**: S3-compatible (MinIO or AWS S3) for raw data archive
- **Auth**: JWT + device-specific API keys per node

Key Services:

1. **Alert Ingestion** (continuous)
   - Polls ALeRCE, TNS, ATLAS, ASAS-SN, AAVSO, Gaia
   - Deduplication + cross-matching
   - Stores in `targets` table
2. **Scoring Engine**
   - Implements the exact scoring function from the PDF (brightness_match × scientific_value × time_criticality × coverage_gap × observability)
   - Runs every 5–15 min + on high-priority alerts
3. **Scheduler Service**
   - Per-node nightly plan generation
   - Greedy priority queue algorithm (visibility windows, exposure estimates, weather)
   - Interrupt handling (nova/kilonova priority)
4. **Node API**
   - `/nodes/{node_id}/plan` (daily download)
   - `/measurements` (POST results)
   - WebSocket for real-time status & interrupts

B. Node Agent (The Heart — Runs on Member's Computer)

Language & Core Library:

- Python 3.12+

- `alpyca` v3.1.2+ (official ASCOM ALPACA Python client — fully maintained, cross-platform, excellent Seestar support)
- `asyncio + aiohttp` for everything non-blocking

Module Structure (proposed package layout):

```
text

boundless_node/
├── agent/
│   ├── main.py                  # entry point + service manager
│   ├── discovery.py             # find Seestar ALPACA devices
│   ├── devices.py               # typed wrappers for Telescope, Camera,
Focuser, FilterWheel
│   ├── executor.py              # executes observation plan
│   ├── image_watcher.py         # watchdog on SMB share
│   ├── photometry/              # local pipeline
│   ├── cloud/                   # communicator + heartbeat
│   ├── safety/                  # park at dawn, connection loss, etc.
│   └── logging.py
├── config/
├── models/                      # Pydantic models for plans, results
├── utils/
└── installer/                   # one-click scripts
```

Key Features Implemented in Detail:

- **Device Management**

  Uses `alpyca Device` classes directly.

  Example (Camera):

```
Python

from alpyca.device import Camera
camera = Camera("http://192.168.1.XXX:32323", 0)
await camera.connect()
await camera.startexposure(duration=180.0, light=True)
```

- **Observation Plan Format** (JSON from cloud)

```
JSON

{
  "night_start": "2026-05-23T02:15:00Z",
  "night_end": "2026-05-23T10:45:00Z",
  "targets": [
    {
      "target_id": "SN2026abc",
      "ra": 210.75,
      "dec": 45.2,
      "exposure_time": 180,
      "frames": 20,
      "filter": "IR",
      "priority": 95,
      "interruptible": true
    }
  ]
}
```

- **Executor Loop** (state machine)
  - Download plan at twilight
  - For each target: slew → plate-solve check (via Telescope) → focus routine → expose stack → filter change
  - Dithering via `Telescope.PulseGuide` (Seestar supports it)
  - Real-time abort handling from cloud
- **Image Handling**
  - Seestar saves FITS to SMB share (standard path)
  - `watchdog + astropy` monitors new files
  - Metadata enrichment (exact RA/Dec, exposure, etc. from ALPACA)

- **Local Photometry Pipeline** (Phase 1)
  - `astropy + photutils + astroquery` (Gaia stars for calibration)
  - Ensemble photometry on comparison stars
  - Outputs: magnitude, error, JD, filter, airmass → tiny JSON
  - Optional: fallback to AstroImageJ batch mode via subprocess
- **Safety & Reliability**
  - Heartbeat to cloud every 60 s
  - Auto-park on cloud disconnect > 10 min or dawn
  - Graceful recovery from Seestar disconnects
  - Local SQLite log of every command/result

C. Member Experience & Installer

- **One-click installer** (PyInstaller + NSIS for Windows, dmg for macOS, .deb for Linux)
- Simple config wizard (IP discovery + API key)
- Optional tiny local web UI (FastAPI + HTMX) for status
- System service (launchd on macOS, systemd on Linux, Windows Service)

3. Communication & Security

- All node ⇔ cloud traffic: HTTPS + JWT
- Optional MQTT for low-bandwidth real-time interrupts
- Nodes never expose ports; cloud initiates most commands

4. Phased Development Roadmap (Practical)

Phase 0 (1 week — you on your Mac)

- Node Agent skeleton + ALPACA connection + basic slew/expose test

Phase 1 (2–3 weeks)

- Full executor + image watcher + basic photometry stub

Phase 2 (3–4 weeks)

- Cloud integration + plan execution + safety

Phase 3

- Advanced photometry + AAVSO formatting

Phase 4

- Optional NINA Advanced API backend (Windows only)
- KStars/EKOS integration (macOS/Linux)

This architecture is **detailed enough to hand to a developer** (or for you to start coding today) while remaining flexible.

Would you like me to:

1. Start writing the actual **Node Agent skeleton code** (with working ALPACA examples you can run on your Mac right now)?
2. Define the exact **Cloud ↔ Node API contracts** (OpenAPI spec)?
3. Or write the **detailed setup guide** for your Mac first?

Just tell me where to begin — I'm ready to go as deep as you want.